Capstone project for the Bachelor's degree at Kutaisi International University

# Hardware Disk Implementation for MIPS CPU
## using SD card flash memory

Author: Demetre Lataria

Thesis advisor: Prof. Dr. Wolfgang Paul

# Declaration

I declare that this thesis has been composed solely by myself and that it has not been submitted, in whole or in part, in any previous application for a degree. Except where stated otherwise by reference or acknowledgment, the work presented is entirely my own.

# Contents


**1 Introduction** **5**
1.1 Abstract . . . 5
1.2 Structure of the thesis . . . 6

**2 Overview of the hardware disk specification** **7**

**3 Overview of the design structure** **8**
3.1 Physical structure of the SD Card . . . 8
3.1.1 Briefly on the SPI protocol . . . 8
3.1.2 SD card wires . . . 9
3.2 SD Card SPI IP . . . 9
3.2.1 Utilization of the SD card IP . . . 10
3.3 Use of On-Chip SRAM . . . 11
3.3.1 Memory structure . . . 12
3.4 The disk interface implementation . . . 13
3.4.1 Overview of the structure . . . 13
3.4.2 Disk interface module . . . 14
3.4.3 State Machine . . . 14

**4 Integration into the MIPS CPU hardware design** **21**
4.1 Full diagram of the MIPS CPU with disk . . . 21
4.2 Example assembly code . . . 22

**5 User guide on utilizing SD card on your personal computer** **24**

**6 Summary** **25**
6.0.1 Technical Implementation . . . 25
6.0.2 Key Features . . . 25
6.0.3 Results . . . 25

# Chapter 1

# Introduction

## 1.1 Abstract

The paper presents the design and implementation of a disk storage interface for **the Microprocessor without Interlocked Pipelined Stages** (MIPS) **Central Processing Unit** (CPU) hardware specification described in **System Architecture: An Ordinary Engineering Discipline**[1] and discussed in the lectures of Introduction to Computer Architecture in "Kutaisi International University". The design targets the **Altera Cyclone V Field Programmable Gate Array** (FPGA) board (Figure 1), which from now on will be refered to as *"the board"*. The work extends previous implementations of the MIPS CPU architecture by integrating a disk storage solution using **Secure Disk** (SD) cards [2] (Figure 2) as the storage medium.

While the board provides hardware support for storage mediums that can be used as the disk storage, official modules or IPs are not provided. The paper examines itegration of unofficial IP by **EmbedFile**[5] and writing a disk interface which adapts design for usage in MIPS machine. The disk interface module was developed by me to comply with established specifications while providing essential storage capabilities for kernel and operating system development on the MIPS platform.

Key contributions include the design of a custom disk interface module that handles SD card IP protocol, the integration of page memory management for disk operations, and the seamless incorporation of the storage subsystem into the existing MIPS CPU implementation. The disk interface is meant to fit together with the following capstone projects:

- Basic compiler
- Improved compiler, assembler, simulator
- Basic hardware: processor core, memory, interrupt support [3]
- Memory Managment Unit

where it will provide support for persistent storage operations.

The paper provides a comprehensive examination of SD card internal memory structures, detailed analysis of the disk interface logic, and thorough documentation of the integration process. Testing and evaluation results demonstrate the successful operation of the storage interface, establishing a foundation for future development of operating systems and kernel implementations on the MIPS FPGA platform.

**Secure Disk** (SD) card [2] (Figure 2) was chosen to act as the physical storage media and the appropriate disk interface module was written by me to adhere specification in the [1]. The SD Card comes in two physical formats: *Full-size* and *micro*. The only difference between the two is the physical size. The board only supports *micro* SD cards, thus in all consequent mentions of the SD card, the *micro* SD card will be implied.

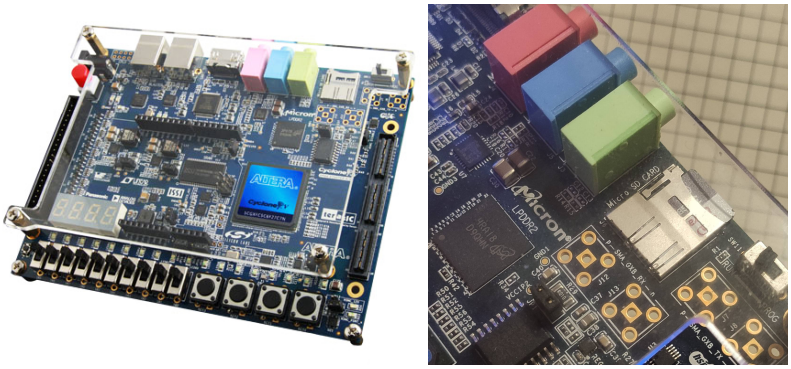


Figure 1 FPGA board and the SD card slot.




Figure 2 Micro SD card.

## 1.2 Structure of the thesis

We will go over the project in the following manner:

- Overview of the underlying specification.
- Minimal explanation of the SD card's internal memory structure.
- Description of the SD Card IP and page memory of the disk interface.
- In-depth look into the disk interface module logic.
- Intagration process into the MIPS CPU[3] and example program.
- Testing and evaluation results.

# Chapter 2

# Overview of the hardware disk specification

The following is the short overview of the specification of the hard disk described by [1].
The disk interface is designed to serve the purpose of the abstraction layer between the physical disk and the MIPS CPU hardware. It mirrors the specification described by [1].

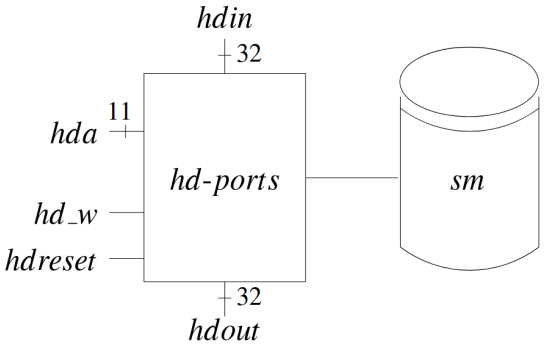



Figure 3 - The interface of the disk from [1].

The specification depicts the Hard Disk ports (hd-ports) that is used to access the swap memory (sp) via the following logic:

- hd-ports is memory mapped onto the two page memory, where the first page is used as the buffer between the CPU and the disk memory. The second page's only first two words are used to store two registers: *Swap memory address register (spar)* and *Command and Status register (cmsr)*.
- *spar* stores 32-bit value which is used to address the pages of the *swap memory*, which the CPU would like to access.
- *cmsr* stores 2-bit value which is used to either select the operation to be performed on the selected page from the *swap memory* or to check the current status of the disk.
  - $cmsr[1:0] = 2'b00$: Disk is ready to recieve commands from the CPU.
  - $cmsr[1:0] = 2'b01$: is set by the CPU when it wants to request read operation.
  - $cmsr[1:0] = 2'b10$: is set by the CPU when it wants to request write operation.
  - Value of the *cmsr* is set to $2'b00$ by the disk when it has finished performing the requested operation. Thus the *cmsr* register can be read to check if the disk has yet to finish the requested operation (If the disk is busy).

The disk interface module will perform as the *hard disk ports (hd-ports)* and *swap memory (sm)* will be represented by the SD card's physical memory.

# Chapter 3

# Overview of the design structure

## 3.1 Physical structure of the SD Card

SD card consists of few components. It has it's own controller inside which acts as an abstraction layer between the user and the physical memory where the data is stored. It also has it's own set of registers where information of the specifics such as operating conditions, identification, status and other information is stored. The detailed specification can be read in [2]. Figure 3 depicts detailed diagram of the structure of the SD card. The physical memory in the SD card is organized in the following matter: SD card's memory is collection of blocks of 512 bytes each. The memory is block addressable and can be addressed up to $2^{32}$ blocks. Read and write operations are done one block per operation. Duration of an each operation depends on the capabilities of the SD card since their working frequency varies significantly depending on the card and the reader.

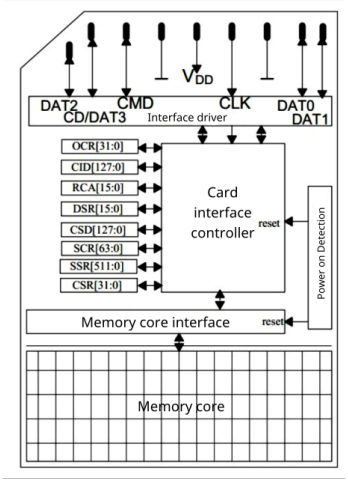



Figure 3 Physical structure of the SD card.

The SD card supports two protocols for interfacing with the board: *Secure Disk Input/Output (SDIO)* and *Serial Peripheral Interface (SPI)*. We will be using SPI.

### 3.1.1 Briefly on the SPI protocol

SPI is a serial protocol used for communicating between multiple devices. While it is not necessary to describe it since IP abstracts it and user will never directly work with it, it is important for understanding connections on the FPGA board. It supports two types of devices: Master and Slave(s). Detailed specification is provided in [4]. We have single master device which coordinates communication and multiple slave devices waiting for master device to initialize communication with them:

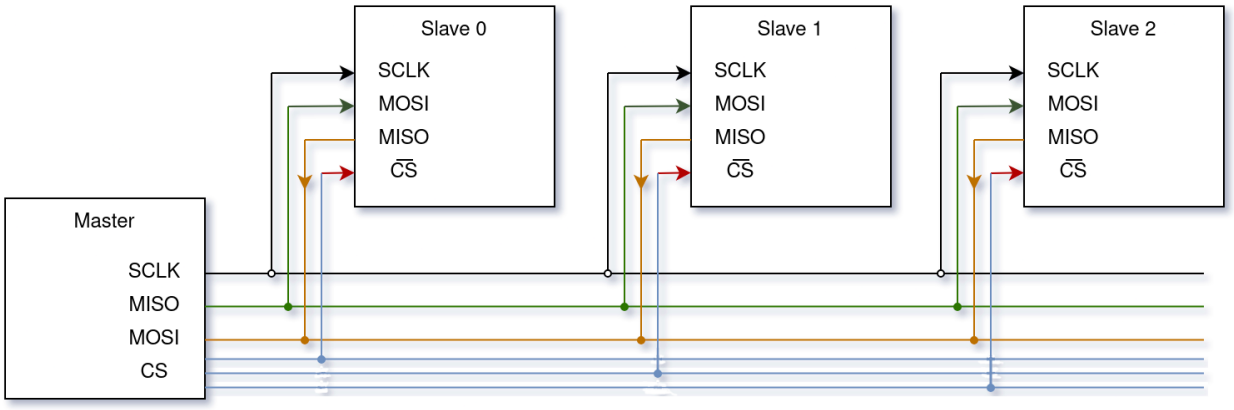



Figure 4 SPI protocol.

The following are signal definitions of the protocol:

- SCK: Serial clock signal. Sent from the master to the slaves.
- SDO/MOSI: Serial data output or Master Out Slave In. One-directional single bit signal that carries data from Master to Slave.
- SDI/MISO: Serial data input or Master In Slave Out. One-directional single bit signal that carries data from Slave to Master.
- SS/CS: Slave Select or Chip Select. Used to choose which Slave device Master is communicating by pulling down (setting signal to low) CS signal.

### 3.1.2 SD card wires

Now that we understand the SPI signals and their functions, we can define the corresponding wires for the SD card. The following is definition of wires on the board via which SD card is connected to the FPGA:

- SD_CLK: Clock input signal. Module requires the clock signal to have a frequency of 50MHz. This clock is driven by the FPGA board.
- SD_CMD: Same as MOSI. Data input of an SD card in SPI mode.
- SD_DAT[3:0]: 4-bit input/output signal. In SPI mode SD_DAT[0] is same as MISO (Data output) and SD_DAT[3] is CS (chip select). SD_DAT[1] and SD_DAT[2] are not used.

SPI is only the way we **carry data** from and to the SD card. However the SD card's controller also has an internal protocol which requires the data to be sent in a specific format mentioned in [2].

## 3.2 SD Card SPI IP

EmbedFire's SD Card IP[5] is used for the sake of simplicity. It uses SPI protocol as mentioned above and provides abstraction for ease of use. The SD Card IP's top module *sd_ctrl* consists of multiple submodules responsible for specific operations:

- sd_init.v is the module used for SD card initialization which is required prior to any read/write operation.
- sd_read.v is a module used to read 512-byte block of data on the SD card from specified address.
- sd_write.v is a module used to read 512-byte block of data to the SD card at specified address.

### 3.2.1 Utilization of the SD card IP

The IP has the following interface:

```
// SD card controller instantiation
sd_ctrl sdcard (
    .sys_clk       (clk),
    .sys_clk_shift (clk_shifted),
    .sys_rst_n     (rst_n),

    // SD card interface
    .sd_miso       (SD_DAT[0]),
    .sd_clk        (SD_CLK),
    .sd_cs_n       (SD_DAT[3]),
    .sd_mosi       (SD_CMD),

    // Write interface
    .wr_en         (wr_en),
    .wr_addr       (sd_addr),
    .wr_data       (wr_data),
    .wr_busy       (wr_busy),
    .wr_req        (wr_req),

    // Read interface
    .rd_en         (rd_en),
    .rd_addr       (sd_addr),
    .rd_busy       (rd_busy),
    .rd_data_en    (rd_data_en),
    .rd_data       (rd_data),

    // Status signals
    .init_end      (init_end)
);
```

The SD card part of the wires must look familiar since we went over them when discussing SD card SPI mode. We will go over the new wires:

- sys_clk_shift: The SD card IP has two clock inputs: sys_clk and sys_clk_shifted, both of which must be connected to a 50 MHz clock. However, sys_clk_shifted must also be phase-shifted by 90 degrees relative to sys_clk. Phase shift refers to the amount of delay applied to the clock signal. This clock is used to drive wires primarily connected to the SD card as specified in [5].
- init_end: wire is low until the SD card successfully initializes and ready to receive read/write commands.

Wires for write operation:

- wr_en: input wire for starting the write operation.
- wr_addr[31:0]: the address of the block that is requested to be written to.
- wr_data[15:0]: 2 bytes of data to be written to the SD card's current block.
- wr_busy: output wire that stays high while the writing to the block is in progress, until the write operation is finished.
- wr_req: As this IP lets you write 2 bytes at the time, this output signal is used by the SD card to request next 2 bytes of data and notify disk interface module to provide it. Data must be provided on the negative edge of this signal.

Here's a waveform diagram for the write operation according to upper description:

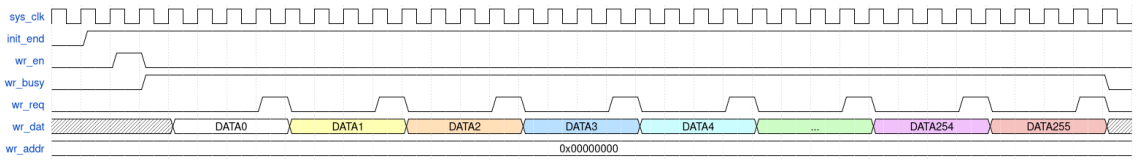



Figure 5 - SD card IP write operation to address 0.

Single write operation write 512 bytes to the SD card. *DAT0* to *DAT255* denote 16 bit data values written to the SD card.

Wires for read operation are named similarly and serve the same purpose as write operation:

- rd_en: input wire for starting the read operation.
- rd_addr[31:0]: the address of the block that is requested to be read.
- rd_busy: same as wr_busy but for read operation.
- rd_data[15:0]: data output read from SD card.
- rd_data_en: similar to wr_req. Toggled by the SD card to notify disk interface that next 2 byte value has been provided. The next 2 bytes are provided on the positive edge of this signal.

Here's a waveform diagram for the read operation according to upper description:

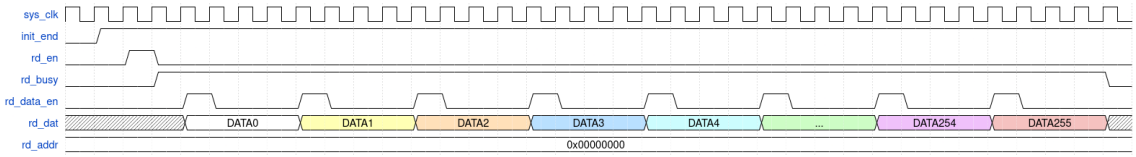



Figure 6 - SD card IP read operation from address 0.

Single read operation reads 512 bytes from the SD card. *DAT0* to *DAT255* denote 16 bit data values read from the SD card.

The following diagram represents state machine of the SD card IP:

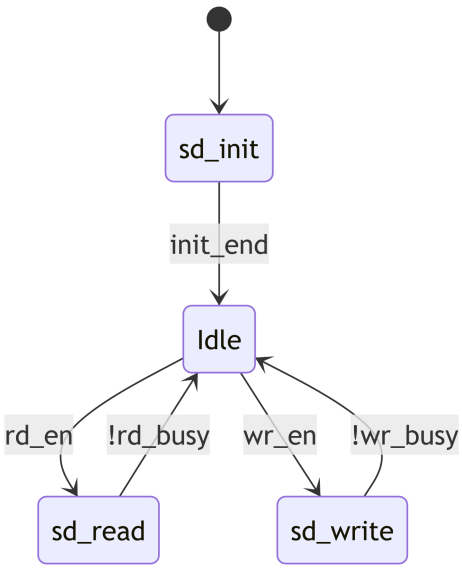



Figure 7 - SD card IP state machine.

## 3.3 Use of On-Chip SRAM

As discussed in the specification of the hardware disk, we require two pages of word addressable memory to map two pages of the hardware disk port. The On-Chip SRAM was chosen for this task. It has following interface with two ports for simultanious access by the disk interface and the CPU:

```
disk_pagemem pagemem (
    .clock(clk),

    // Port for the disk
    .address_a(disk_sram_addr),
    .data_a(disk_sram_data_in),
    .byteena_a(disk_sram_byteena),
    .wren_a(disk_sram_wren),
    .q_a(disk_sram_data_out),

    // Port for the processor
    .address_b(hd_a),
    .data_b(hdin),
    .wren_b(hd_w),
    .q_b(hdout)
);
```

wires ending with *_a* are exclusivly used by the disk interface, while wire ending with *_b* are exclusivly used by the CPU.

- clock: Clock input signal (50MHz).
- data_a/data_b[31:0]: 32-bit data input.
- byteena_a[3:0]: Byte enable input used only by the disk interface. Controls which bytes of the *data_a* are written to the SRAM.
- wren_a/wren_b: Write enable signal.
- q_a/q_b[31:0]: 32-bit data output.

### 3.3.1 Memory structure

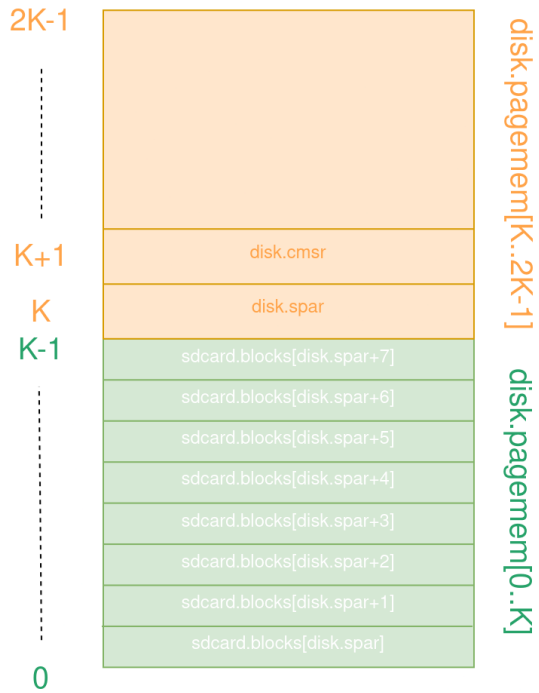


Figure 8 - The structure of the two page memory of the disk module.

Let's introduce some definitions to represent the overall structure of the page memory.

Let $K = 1024$ denote size of the single page in words. Let $disk.pagemem$ represent two page memory of the disk interface and $sdcard.blocks$ be a block memory of the SD card. Then:

$$disk.spar = pagemem[K]$$

*spar* is stored in first word of the second page in disk's page memory or $K$th word in the two page memory.

$$disk.cmsr = disk.pagemem[K + 1]$$

*cmsr* is the second word of the second page in disk's page memory or $K + 1$th word in the page memory.

$$disk.pagemem[0..K - 1] = sdcard.blocks[spar..spar + 7]$$

Since each block in the SD card is 512 bytes or 128 words, the first page of the disk interface will consist of $1024/128 = 8$ consequent blocks. This can be observed in the figure 6.

## 3.4 The disk interface implementation

The disk interface module is an abstraction written by me in Verilog for adapting SD card to the MIPS CPU.

### 3.4.1 Overview of the structure

The following figure depicts the simplified structure of the disk interface:

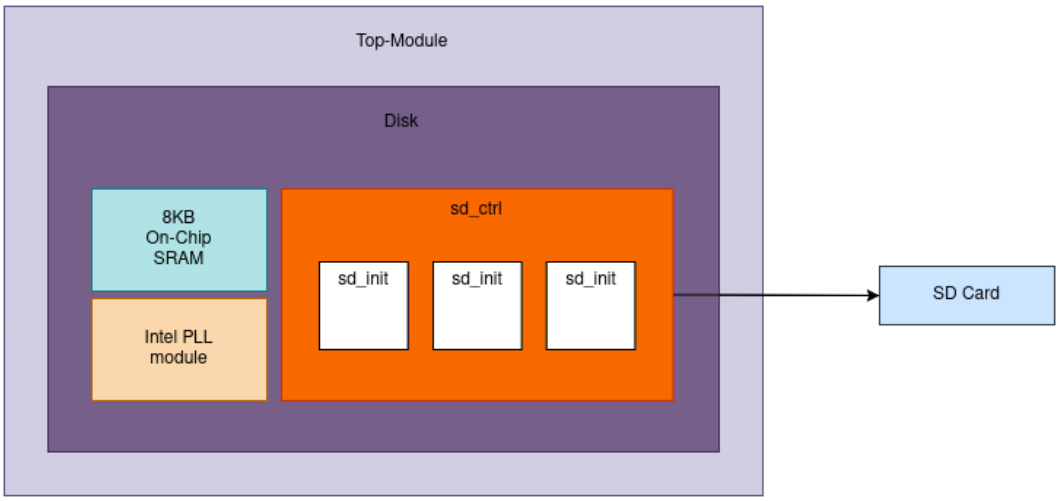



Figure 9 - disk interface module structure overview.

The Disk interface module encapsulates following modules:

- 8KB On-Chip Static Random Access Memory (SRAM) as two page memory: On-Chip memory is 8 Kilobytes of word addressable memory generated inside the FPGA provided by Intel and specified by them in [6]. Its use in disk interface will be discussed shortly.
- Intel PLL(Phase-Locked Loop) module[7]: A PLL is an electronic circuit that synchronizes an output signal's phase with a reference signal, allowing it to generate a stable clock signal with a specific frequency and phase relationship. It is recommended by Intel for stable clock generation. In this context, the PLL is used to provide the SD card IP with a 90-degree phase-shifted clock signal (*sys_clk_shifted*).
- sd_ctrl: The main module of the SD card IP provided by [5] as discussed before.

Disk interface module accesses SD card exclusively through the sd_ctrl module.

### 3.4.2 Disk interface module

```
disk port(
    .clk(clk),

    .rst_n(rst_n),
    .hda(disk_hda),
    .hd_w(disk_hd_w),
    .hdin(disk_hdin),
    .hdout(disk_hdout),

    .SD_CLK(SD_CLK),
    .SD_CMD(SD_CMD),
    .SD_DAT(SD_DAT),
);
```

The disk interface module consists of the following wires:

- hdin[31:0]: 32 bit data input. Used to write data to the disk interface's page memory.
- hdout[31:0]: 32 bit data output. Used to read data from the disk interface's page memory.
- hda[10:0]: 11 bit address input. Used to address data of the disk interface's page memory. ($2^{11} = 2048$ words or $8KB$ addressable).
- hd_w: write enable input for the page memory of the disk interface.
- hdreset: reset signal.

alongside with additional wires required for connecting SD card and clock signal to the interface:

- clk: Clock input signal.
- SD_CLK: Clock output for SD card.
- SD_CMD: explained in the previous section.
- SD_DAT[3:0]: explained in the previous section.

### 3.4.3 State Machine

The following figure represents diagram of the state machine of the disk interface.

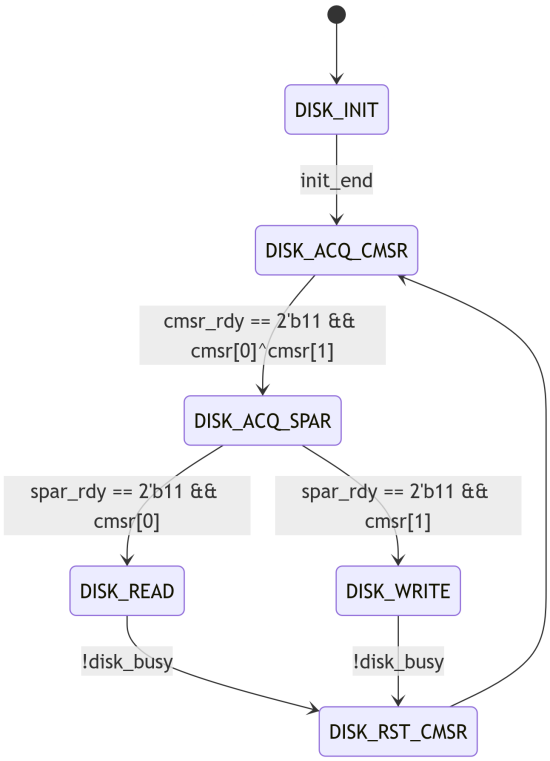



Figure 10 - disk interface state machine.

- DISK_INIT: Disk is waiting for SD card to initialize.
- DISK_RST_CMSR: Disk is reseting value of *cmsr* in the page memory to notify that disk is no longer busy and is ready to perform next operation.
- DISK_ACQ_CMSR: Disk is aquiring the value of *cmsr* from the page memory.
- DISK_ACQ_SPAR: Disk is acquiring the value of *spar* from the page memory.
- DISK_READ: Disk is reading data from SD card and writing to the first page of the page memory.
- DISK_Write: Disk is reading from the first page of the page memory and writing data to the SD Card.

Verilog code for the state machine:

```
localparam [2:0]
    DISK_INIT      = 3'h0,  // Wait for SD card to initialize
    DISK_RST_CMSR  = 3'h1,  // Request reset value of CMSR
    DISK_ACQ_CMSR  = 3'h2,  // Recieve CMSR value
    DISK_ACQ_SPAR  = 3'h3,  // Request SPAR value
    DISK_READ      = 3'h4,  // Read Operation
    DISK_WRITE     = 3'h5;  // Write Operation

reg [2:0] state = DISK_INIT;
reg [2:0] next_state;

...

always @(*) begin
    next_state = state;
```

```
16
17      case (state)
18          DISK_INIT: begin
19              next_state = init_end ? DISK_RST_CMSR : DISK_INIT;
20          end
21
22          DISK_RST_CMSR: begin
23              next_state = DISK_ACQ_CMSR;
24          end
25
26          DISK_ACQ_CMSR: begin
27          next_state = (cmsr_rdy == 2'b11) && (cmsr[0] ^ cmsr[1]) ? DISK_ACQ_SPAR :
               DISK_ACQ_CMSR;
28          end
29
30          DISK_ACQ_SPAR: begin
31              if(spar_rdy == 2'b11)
32                  next_state = cmsr[0] ? DISK_READ : DISK_WRITE;
33                              else
34                  next_state = DISK_ACQ_SPAR;
35          end
36
37          DISK_READ: begin
38              if (!disk_busy)
39                  next_state = DISK_RST_CMSR;
40          end
41
42          DISK_WRITE: begin
43              if (!disk_busy)
44                  next_state = DISK_RST_CMSR;
45          end
46
47          default: begin
48              next_state = DISK_INIT;
49          end
50      endcase
51  end
```

The disk interface has an *disk_busy* signal which is high while the SD card is currently processing an operation or all 8 blocks haven't been processed.

$$disk_busy = rd_en|rd_busy|wr_en|wr_busy$$

Busy signals from the SD card are self explanatory. SD card signals for starting read or write operation are included, since we have to process multiple blocks. For the SD card the operation on the block will be finished, thus it's busy signals will be set to low, however that doesn't necessarily mean that we have finished processing all the blocks. *rd_en* and *wr_en* will be active until the operation on the last block has begun.

Now let's look into the each operation.

### Acquisition of required values

Before we start analysis of the read and write operations, first we need to discuss handling *cmsr* and *spar* register values. First and foremost, The disk interface waits for the sd card to initialize. While this is happening it stays in the *DISK_INIT* state.

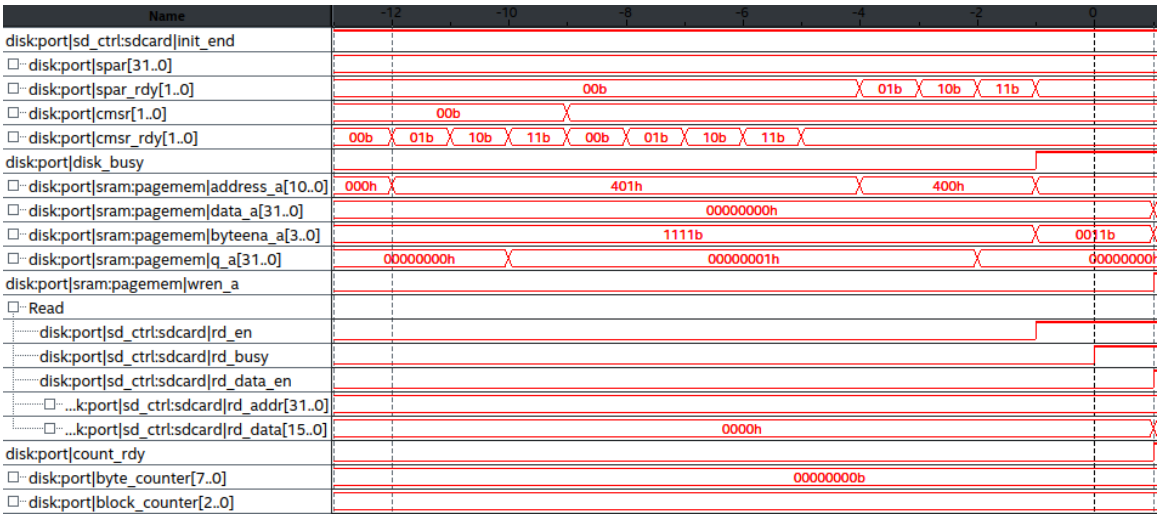



Figure 11 - Waveform diagram for acquiring the register values.

Next the disk interface moves to the *DISK_ACQ_CMSR*. The interface sets address in of the SRAM page memory to the location of the *cmsr* register which in hex will be $0x801$ (first word of the second page) and sets write enable of the SRAM low to read from the address. Since the SRAM module requires multiple cycles to give the disk the value stored in the requested address, we use *cmsr_rdy* 2-bit register as counter to count 3 clock cycles: 1 for setting the wires of the SRAM, 2 for delay of the SRAM. After the acquisition the value is checked for either $2'b01$ or $2'b10$ (read/write operation request). If one of the operations are requested we move onto the acquisiton of the value of the *spar* register, otherwise we stay in *DISK_ACQ_CMSR* and keep reading the value.

In *DISK_ACQ_SPAR* we perform similarly as with the *cmsr* register. Disk sets address in of $0x800$ to the SRAM *spar* and sets write enable low for read. *spar_rdy* is used exactly the same as *cmsr_rdy*. Since there are no special conditions for the value of the *spar* register, we acquire the value and move onto the requested disk operation.

### Read Operation

#### Begin read

To begin the read operation the disk interface raises the *rd_en* signal. The signal is required to be high for at least 1 clock cycle. Since we have to read one page which is equivalent to 8 blocks, we will keep it high until the read operation of the last block has begun. This sets *rd_busy* high by the SD card until the single block has been read. The interface is notified of the arrival of the first 16-bits by the *rd_data_en* signal issued by the SD card. The value is available until the next posedge of the *rd_data_en* signal. The interface directs the read data to its page memory's first page. Since SD card sends the interface 16-bits at the time and its page memory is word addressable, the interface uses *byte_counter* and *block_counter* to keep track of where the data must be written in the page memory. *byte_counter* is 8-bit value (256 half-words per block) that keeps track of how many half-words have been read so far, while *block_counter* is 3-bit value (8 blocks per page) that keeps track of which block of the sd card is currently read. the interface sets the data input signal of the page memory to the half-word concatinated to itself: $(rd_data, rd_data)$ and uses *byte_enable* wire of the page memory to control which bytes of the data input to write to it.

```
1 assign sd_addr = spar + block_counter; //SD card block address
2 ...
3 //SRAM signal assignments
4 assign disk_sram_data_in = (state == DISK_READ) ? {rd_data, rd_data} : sram_reg_data;
5 assign disk_sram_byteena = ((state == DISK_READ) || (state == DISK_WRITE)) ?
     disk_sram_byteena_incr : 4'b1111;
6 assign disk_sram_wren = (state == DISK_READ) ? rd_data_en : sram_reg_wren;
7 assign disk_sram_addr = ((state == DISK_READ) || (state == DISK_WRITE)) ?
8     {block_counter, byte_counter[7:1]} : sram_reg_addr;
```

*byte_enable* is controlled by the *byte_counter* accordingly: $byte_enable[3:0] = byte_counter[0]?4'b0011 : 4'b1100$. Thus the *rd_data* is written to the most significant 2 bytes of the page if the *byte_counter* is even and to the least significant 2 bytes if the counter is odd. For setting the address of the page memory (which address of the page to write the data to) concatenation of the block_counter and 7

most significant bits of the byte_counter (since it counts half-words and we are interested in words) is used:

$$old_value = disk.pagemem[(block_counter, byte_counter[7:1])];$$

$$\begin{aligned} &disk.pagemem[(block_counter, byte_counter[7:1])] = \\ &(byte_enable[3]?rd_data[15:8] : old_value[31:24], \\ &byte_enable[2]?rd_data[7:0] : old_value[23:16], \\ &byte_enable[1]?rd_data[15:8] : old_value[15:8], \\ &byte_enable[0]?rd_data[7:0] : old_value[7:0]); \end{aligned}$$

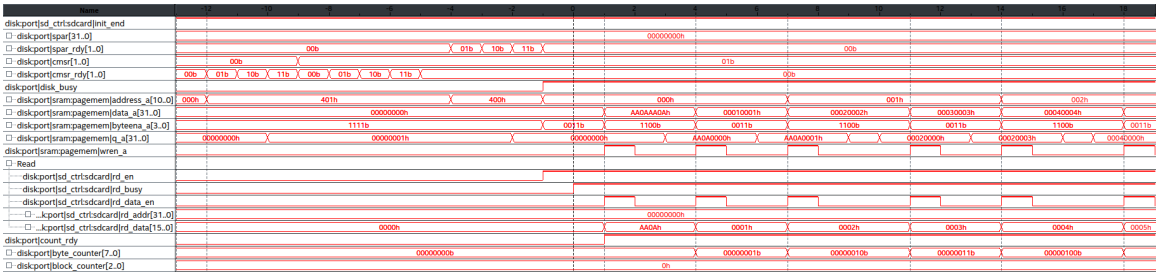


Figure 12 - Begining of the read operation.

**Moving to the next block**

The *rd_busy* signal is set to low by the SD card to denote the end of the read operation. However we have read only first block of the SD card which is $1/8$th of the page. For this reason we kept the *rd_en* signal high. SD card responds to the *rd_en* signal and starts reading the next block. We set the address of the SD card block to the *block_counter*'s value which is incremented on each negative edge detected in the *rd_busy* signal. We continue according to the process above.

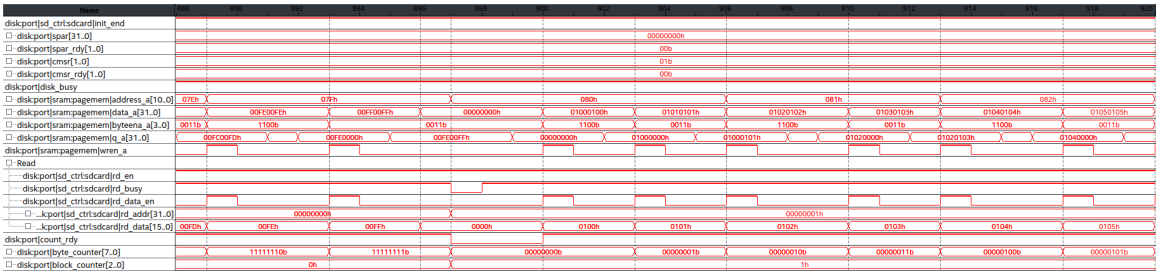


Figure 13 - End of reading block 0 and begining of reading block 1.

**Finishing the operation**

Once the *block_counter* reaches the value of 8 (operation on the last block is about to begin), the interface is required to set the *rd_en* signal after the read operation of the last block has begun and before it has finished to avoid processing extra block. The interface sets the *rd_en* to low after we have read half of the block by checking the value of *byte_counter*. At the end of this operation, the *disk_busy* signal is set to low, since *rd_en* signal is already low and SD card sets the *rd_busy* signal low each time operating on the single block has been finished. The interface goes to *DISK_RST_CMSR* state to set the *cmsr* register to $2'b00$, which notifies CPU that the read operation has been finished. Next the interface moves to *DISK_ACQ_CMSR* state to keep checking until the next request from the CPU.

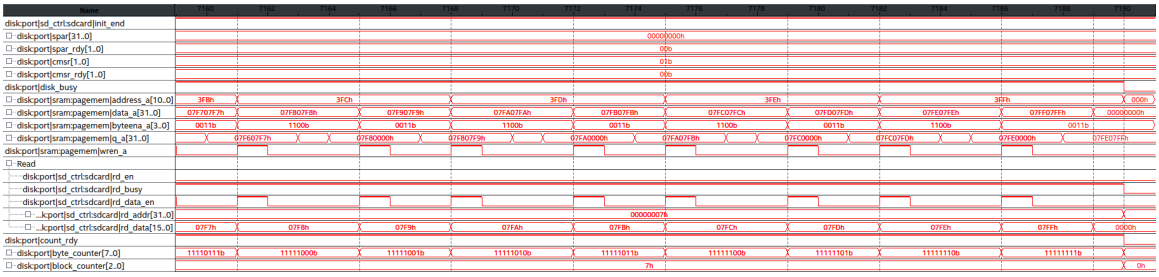


Figure 14 - End of the read operation.

### Write Operation

Since both read and write operations are similar, we will only go over the differences.

#### Begin write

The interface raises the *wr_en* signal. This sets *wr_busy* high by the SD card. The interface reads word from the first page of its page memory and passes it to the SD card:

$$disk_sram_data_out = disk.pagemem[(block_counter, byte_counter[7:1])]$$

$$wr_data = byte_counter[0] ? disk_sram_data_out[0:15] : disk_sram_data_out[16:31]$$

Data must be provided to the SD card at the negative edge of the *wr_req* signal until the next negative edge of the signal.

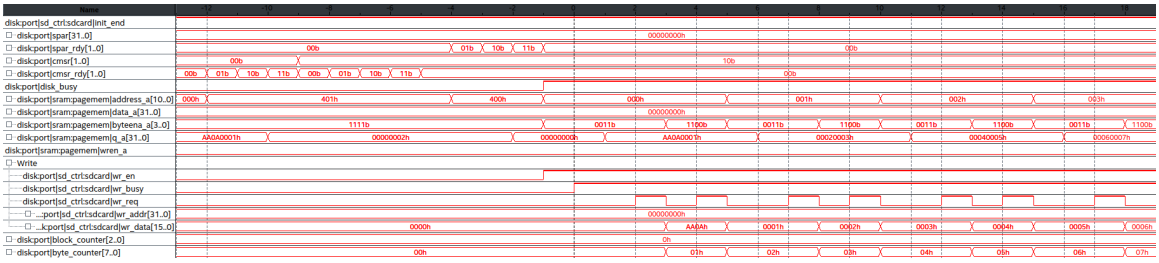


Figure 15 - Begining of the write operation.

#### Moving to the next block

No difference compared to the read operation.

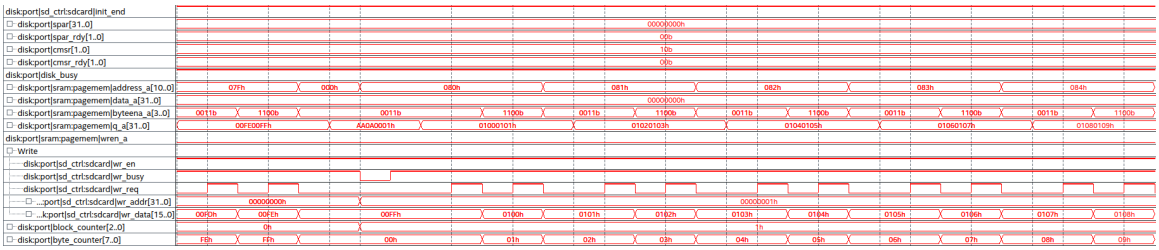


Figure 16 - End of writing block 0 and begining of writing block 1.

#### Finishing the opeartion

No difference compared to the read operation.

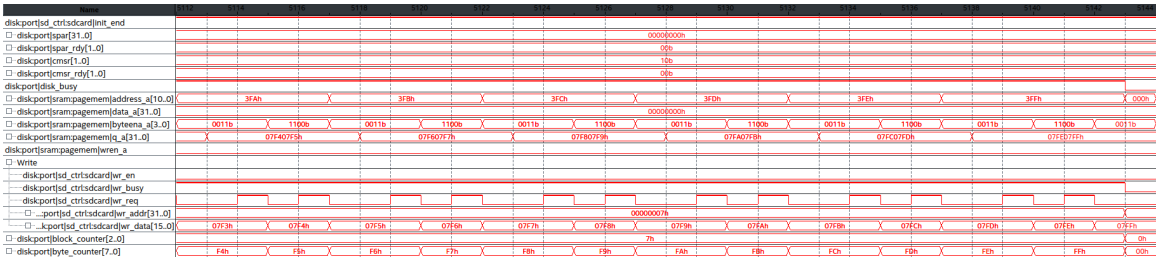


Figure 17 - End of the writing operation.

### Implementation of counters

We have two wires that switch between the read and write wires of the SD card. *op_req* has value of either *rd_data_en* or the negation of the *wr_req*, since we are interested in negative edges of the write request signal as uppermentioned. *op_busy* is the busy signal of the operation (*rd_busy* or *wr_busy*).

```
wire op_req = (state == DISK_READ) ? rd_data_en : (!wr_req);
wire op_busy = (state == DISK_READ) ? rd_busy : wr_busy;
```

Counting the current block is simple. We increment the *block_counter* each time the negative edge of the operation's busy signal was detected (each time the SD card finished operation).

```
always @(negedge op_busy or negedge rst_n) begin
                if(!rst_n)
                        block_counter <= 0;
                else
                        block_counter <= block_counter + 1;
end
```

The following implements the *byte_counter*, alongside with logic for handling transformation of half-words to words and the opposite. *count_rdy* register is used to delay the disk read operation by single clock cycle (In this case clock signal is *op_req*). This is done to start indexing of the half-words from 0 instead of from 1. If we remove the delay, on the positive edge of the *op_req*, *byte_counter* will increase by one altering the address to which the first half-word should have been written.

```
reg count_rdy = 0;
always @(posedge op_req or negedge op_busy or negedge rst_n) begin
        if (!rst_n) begin
                byte_counter <= 8'h0;
                wr_data <= 16'h0;
                disk_sram_byteena_incr <= 4'b0011;
                count_rdy <= 0;
        end
        else if (!op_busy) begin
                count_rdy <= 0;
                byte_counter <= 8'h0;
                disk_sram_byteena_incr <= 4'b0011;
        end
        else if (op_req) begin
                if((count_rdy | wr_req) && ({block_counter, byte_counter} != 11'h7FF))
                        byte_counter <= byte_counter + 1;

                if(~count_rdy)
                        count_rdy <= 1;

                wr_data <= byte_counter[0] ?
                   disk_sram_data_out[15:0] : disk_sram_data_out[31:16];
                disk_sram_byteena_incr = disk_sram_byteena_incr[0] ?
                    disk_sram_byteena_incr << 2 : disk_sram_byteena_incr >> 2;
        end
end
```

# Chapter 4

# Integration into the MIPS CPU hardware design

## 4.1 Full diagram of the MIPS CPU with disk

The following is the extension of the diagram provided by [3] with connected disk module:

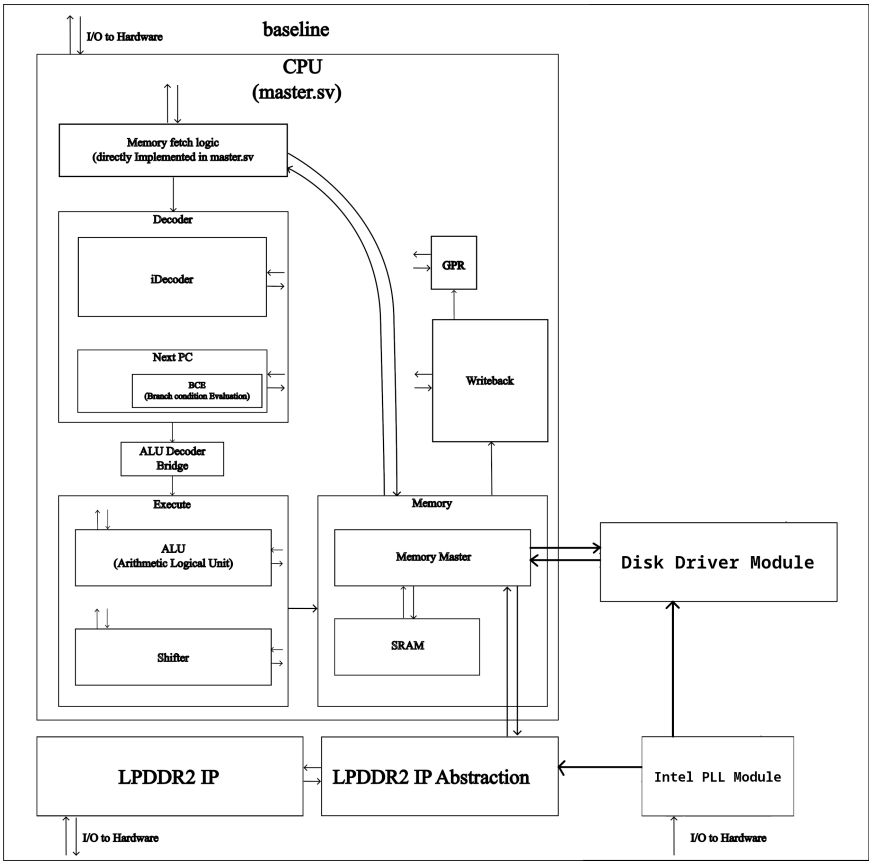



Figure 19 - MIPS CPU with connected disk module.

The disk module is connected to the master memory in the same manner as the SRAM and LPDDR2 module (described by [3]). The LPDDR2 module has its own Intel PLL module integrated inside of its IP module. Quartus Software has limitations for the reference clock input for these modules. For this reason disk module's PLL clock output was used for the LPDDR2 module's PLL reference clock input. Thus the PLL module was put outside of the disk module to be connected both to the LPDDR2 and disk modules.

After adding the disk, the memory addressing of the CPU changed as following:

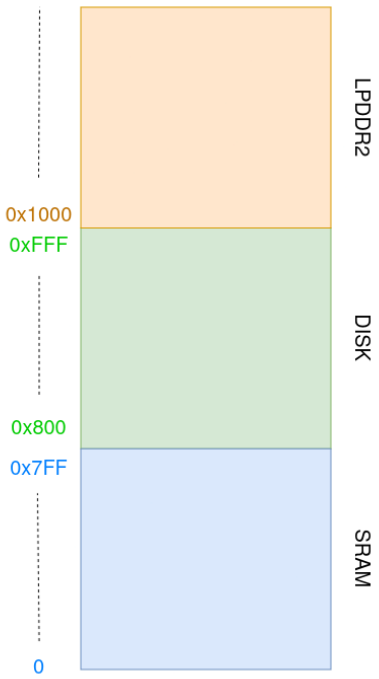



Figure 20 - MIPS CPU Memory Layout after adding the disk.

## 4.2 Example assembly code

The MIPS CPU project with the disk interface module can be acquired from GitHub Repository: which already comes with the example program for the disk. For example program, we save the following assembly code to the SD card from the computer, which sums up all the numbers from 1 to 100:

```
main:
    li $t0, 1           # counter = 1
    li $a1, 0           # sum = 0 (register 5)
    li $t1, 100         # upper limit = 100

loop:
    add $a1, $a1, $t0   # sum = sum + counter
    addi $t0, $t0, 1    # counter = counter + 1
    ble $t0, $t1, loop # if counter <= 100, continue loop
```

After we will insert the SD card into the board with CPU executing following assembly code on the power up of the board, which will give the disk interface command to read from the SD card, copy read data from the disk buffer to the SRAM, and jump to the SRAM to execute:

```
main:
   li $t0, 0x3004
   li $t1, 1
   sw $t1, 0($t0)

check_loop:
   lw $t2, 0($t0)
   bnez $t2, check_loop

```

```
    li $t3, 0x2000
    li $t5, 0x1000
    li $t6, 25

copy_loop:
    lw $t4, 0($t3)
    sw $t4, 0($t5)

    addi $t3, $t3, 4
    addi $t5, $t5, 4
    addi $t6, $t6, -1

    bnez $t6, copy_loop

    li $t7, 0x1000
    jr $t7
```

The following demonstrates the program binary loaded from the SD card and results of the execution:

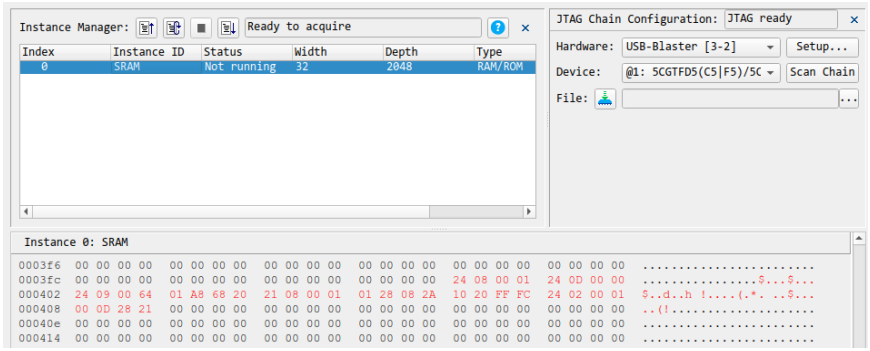



Figure 21 - program loaded from disk to SRAM of the CPU.

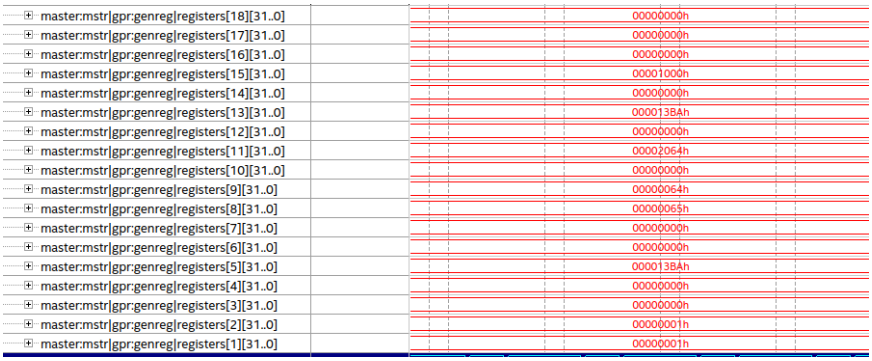



Figure 22 - Result of the execution and values of all the registers. Result is stored in the register $5.

The CPU has finished executing with the result 0x13BA which in decimal is 5050 on register $5.

# Chapter 5

# User guide on utilizing SD card on your personal computer

It can be useful to preload your SD card with programs or data for the board to process, or to check what the board wrote to the SD card. I will provide example for Linux computer. This is also possible on Windows computers using the HxD Hexadecimal editor program. You can learn more about it and how to access SD card or other disks for editing or reading purposes on their website: . On linux we use *dd* program, which is a command-line low-level data copying and conversion utility. Execute following command in your terminal with root permission to read from the SD card and save read data to a file:

```
dd if=/dev/mmcblk0 of=output_file bs=4096 count=1 skip=0
```

This command will read from the SD card */dev/mmcblk0 count* amount of *bs=4096* or 4 Kilobytes of data starting from page address (4 kilobyte page) *skip*.

You can view or edit this file with hexadecimal editor of you choice. After you edit or create new file with your editor, you can write the data back to the SD card using the following command:

```
dd if=input_file of=/dev/mmcblk0 bs=4096 count=1 seek=0
```

This command will write to the SD card */dev/mmcblk0 count* amount of *bs=4096* or 4 Kilobytes of data at the page address *seek*.

# Chapter 6

# Summary

This thesis implements a hardware disk storage solution for a MIPS CPU using SD card flash memory on an Altera Cyclone V FPGA board. The project extends existing MIPS CPU architecture to support non-volatile storage, providing a foundation for operating system development.

### 6.0.1 Technical Implementation

The system uses SD cards in SPI mode as the storage medium, with memory organized in 512-byte blocks. A custom disk interface manages communication between the CPU and SD card through a two-page memory architecture:

- **Page 1 (4KB):** Data buffer equivalent to 8 SD card blocks
- **Page 2 (4KB):** Control registers (SPAR for addressing, CMSR for commands/status)

The disk interface operates via a state machine handling initialization, command acquisition, and read/write operations. Key components include 8KB dual-port on-chip SRAM, Intel PLL for clock management, and a third-party SD card controller IP.

### 6.0.2 Key Features

- **Memory-mapped I/O:** Disk appears as standard memory to the CPU
- **Asynchronous operation:** State machine manages multi-cycle SD card operations
- **Concurrent access:** Dual-port memory allows simultaneous CPU and disk operations

### 6.0.3 Results

The implementation successfully demonstrates disk functionality through assembly programs that read/write data and execute code stored on SD cards. The system integrates seamlessly with existing MIPS CPU architecture. This work provides essential storage capabilities for the MIPS CPU platform, enabling future development of file systems and operating systems.

# Bibliography


[1] Wolfgang J Paul, Christoph Baumann, Petro Lutsyk, and Sabine Schmaltz. *System Architecture: An Ordinary Engineering Discipline*. Springer, 2016.

[2] Secure Disk Association. "SD Host Controller Simplified Specification". In: (2018). URL: https://www.sdcard.org/downloads/pls/.

[3] Giorgi Bakradze, Sandro Lortkipanidze, and Lado Turmanidze. "Implementation of a MIPS CPU, Its Peripherals, and Interrupts". Bachelor's Thesis. Kutaisi International University, 2026.

[4] MicroChip. "Serial Peripheral Interface". In: (2007). URL: https://ww1.microchip.com/downloads/en/DeviceDoc/39699b.pdf.

[5] EmbedFire. "SD card data read and write control". In: (2020). URL: https://doc-embedfire-com.translate.goog/fpga/altera/ep4ce10_pro/zh/latest/code/sd.html?_x_tr_sl=auto&_x_tr_tl=en&_x_tr_hl=en-US.

[6] © Intel Corporation. "On-Chip RAM/ROM IP Core". In: (). URL: https://www.intel.com/content/www/us/en/docs/programmable/683130/22-1/on-chip-memory-ram-and-rom-intel-fpga-ip.html.

[7] © Intel Corporation. "Intel Phase-Locked Loop IP Core". In: (). URL: https://www.intel.com/content/www/us/en/docs/programmable/683359/17-0/functional-description.html.